AMAZON ML CHALLENGE 2026 · BUSINESS ENTITY RESOLUTION

Runtime Terrors Logbook

How we went from a 0.799 rule baseline to a 0.979 holdout model in three days on one 8 GB laptop: every version, every decision, what failed, and how we kept the memory problem under control.

Team Sridharan Kannan (lead), Arwindh C N, Raghav V S

Window 25 Sep 00:00 to 27 Sep 23:59 IST

Machine MacBook, Apple M2, 8 GB RAM

Our F0.5, rule → v8	Public leaderboard	Candidates per S1	Peak RAM, blocking
0.799 → 0.979	0.944 → 0.967	50 → 3.9	8.6 → 3.7 GB
	v8 not uploaded yet		

01

The task

Three sources of business records with no shared IDs. Source 1 (S1) is deduplicated. For every S1 business we had to find all matching records in Source 2 and Source 3: zero, one or many. Each record has a name, an address and a country.

The score is **F0.5 per S1 entity, then averaged**. Precision counts about twice as much as recall, so a wrong merge costs more than a missed match. An S1 with no true matches scores 1.0 only if we predict nothing for it.

What we measured on day 1

- Train: 2.21M S1, 5.03M S2, 5.29M S3 (US 60%, India 40%).
- Test: 1.73M S1, 4.89M S2, 5.08M S3, and **France** (15%), which never appears in train.
- Only 5.6% of S1 have no match; on average 3.46 matches each.
- No S2/S3 record belongs to more than one S1 (checked on all 7.6M true pairs).
- 23.5% of India S2 and 13.2% of India S3 names are in Indian scripts. S1 names never are.

Rules we worked under

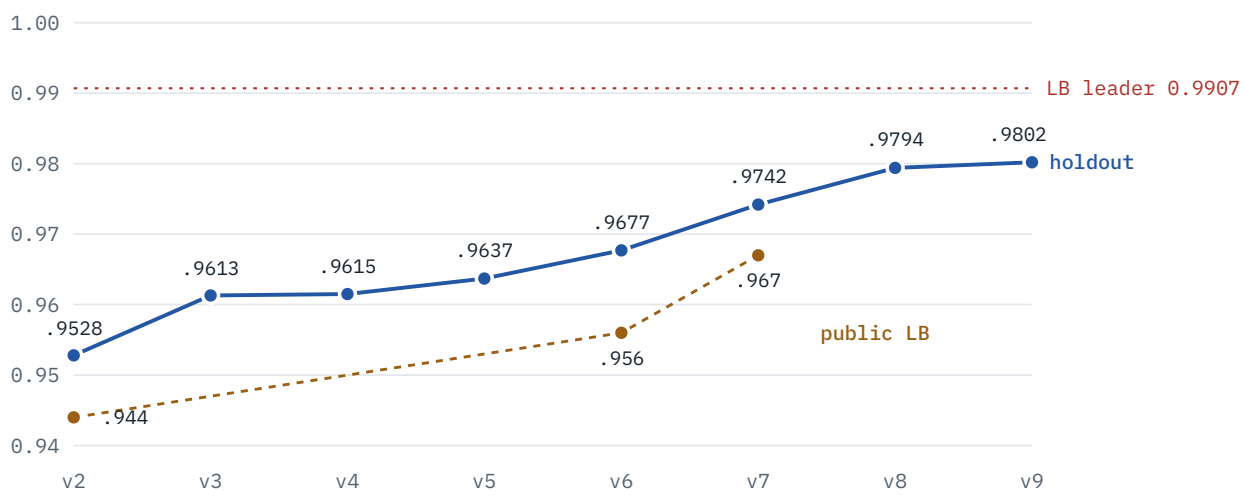
- No external data: no APIs, geocoding, registries or downloaded lists.
- Models must be MIT or Apache 2.0, at most 8B parameters.
- Code must reproduce the outputs from raw data.
- Dataset files are never modified.
- Five leaderboard uploads per day.
- Late update: smaller candidate sets rank higher.

Two output files per submission: `matching_results.tsv` (scored) and `candidate_pairs.tsv` (the candidate set the final model scored).

02

Scores over time

Holdout is 50,000 training S1 entities we never tuned on. It predicted the direction of every leaderboard result.



v1 (rule baseline, val 0.799) sits far below this scale and is left out. v8 and v9 have no leaderboard score yet. Vertical scale: 0.01 per grid line.

The leaderboard always came in below holdout: −0.009 for v2, −0.012 for v6, −0.007 for v7. Our best explanation is France, which the model never saw in training, plus a test set

with more look-alike records per business.

03

Timeline

Times are IST, taken from our commit history and run logs.

Day 1 · Thursday 25 September

- `evening` Read the problem statement and rules. Ran `explore.py` for the statistics above. Loading all three train files at once peaked near 6 GB of the 8 GB.
- `21:17` Moved the project to `Desktop/amazon-ml-challenge-2026.nosync` (".nosync" keeps iCloud from syncing gigabytes), created a Python 3.14 virtual environment and a local-only git repository.
- `21:18` Converted every TSV to Parquet once, verified identical. Loading a big file dropped from about 6 s to 0.4 s.
- `21:28` Built the validation split (train 300k / val 50k / holdout 50k S1 entities, by entity, seed 42, stratified by country, with leakage checks) and an exact F0.5 scorer that reproduces the README example (0.714).
- `21:44` Blocking v1. It worked but used 7.9 GB for 50k queries, peaking at 8.6 GB.
- `22:13` Blocking v2: rarity-based keys and a packed integer index. Recall 0.953 at 50 candidates per S1, peak memory 3.7 GB.

Day 2 · Friday 26 September

- `00:14` Rule baseline tuned on val: 0.799. First full test run (25 min), validated at 01:00.
- `01:47` LightGBM v2 with 38 features and an expected-F0.5 decision: val 0.952, holdout 0.953. Test run 73 min. **First upload: 0.944.**
- `10:36` v3: a learned Indian-script → Latin dictionary (1,305 words, from training pairs only) and "how many S1 share this address" features, after seeing that France shares addresses twice as often. Holdout 0.961.
- `11:43` v4 cascade: a learned filter cut candidates from 50 to 3.9 per S1 with no loss in score. This answered the organisers' update that smaller candidate sets rank higher.
- `12:49` v5: name-uniqueness features. Holdout 0.9637.

`13:53` v6a: character 3-gram similarity (+0.0012). A 3-model ensemble gave nothing and was dropped.

`14:10` v6b: reverse competition, the idea from Sridharan's notebook. Holdout 0.9677. **Upload: 0.956.**

`15:02` Code review of v7 for leakage and memory before running it.

`17:15` Found the "hub" bug: one S1 business collected 142,645 junk candidates through the word "com". Capped reverse-found candidates at 20 per S1.

`17:37` v7: bidirectional blocking. Holdout 0.9742. Test run 93 min. **Upload: 0.967.**

`~20:00` Transformer experiment on half the data: +0.0047 holdout. You approved the "big swing": PyTorch, transformers and a small Apache-2.0 model.

`20:38` v8 started: fine-tuning the transformer in two halves, about 37 minutes each on the Mac GPU.

`22:22` v8 trained: val 0.9792, holdout 0.9794. Smoke test passed; full test run started at 22:31.

Day 3 · Saturday 27 September

`04:57` v8 test run finished after 6.4 hours (planned: 3.5). Official validator PASS. v8 copied to `output/`.

`05:51` v9 recall gate passed: true pairs reaching the candidate lists rose from 0.9706 to 0.9752.

`06:42` v9 trained: val 0.9798, holdout 0.9802 (+0.0008 over v8). Test run started at 06:53.

`09:30` India phase started. Memory filled up and the Mac swapped heavily; batches slowed from about 45 s to 80–100 s.

`10:02` Built a v9 package for Sridharan to run on an AWS GPU server.

`10:30` Built `Runtime_Terrors_submission.zip` (352 MB) with v8 as the final.

`10:32` The v9 test run stopped at batch 225 of 347 when its parent background task was stopped. Its results were only in memory, so none were saved.

`today` AWS GPU quota request still pending; credits available but no GPU server allowed yet.

04

Version by version

VERSION	WHAT CHANGED	CANDS / S1	VAL	HOLDOUT	LB
v1	Rules: name similarity ≥ 0.8, address ≥ 0.6, house numbers agree	50	0.799	–	–
v2	LightGBM on 38 pair and group features; expected-F0.5 decision	50	0.9520	0.9528	0.944
v3	Learned transliteration; number containment; shared-address context	50	0.9610	0.9613	–
v4	Cascade: learned candidate filter before the final model	3.87	0.9621	0.9615	–
v5	S1 name-uniqueness counts	3.79	0.9644	0.9637	–
v6	Character 3-grams + reverse competition	3.79	0.9688	0.9677	0.956
v7	Bidirectional blocking + hub cap	3.84	0.9747	0.9742	0.967
v8	+ fine-tuned transformer score as a feature (final)	3.84	0.9792	0.9794	not uploaded
v9	+ reverse top-10 instead of top-3 (test run not finished)	3.91	0.9798	0.9802	–

v1 and v2: from rules to a learned model

The rule baseline showed the core difficulty: singletons have planted look-alikes with the same name and street but a different house number. Rules got only 58.8% of singletons right. LightGBM with **group features** (how a candidate compares with the other candidates of the same S1) fixed most of that and jumped to 0.952.

v3: languages and France

Indian-script names are phonetic spellings of English words ("ड्रीम बिजनेस प्राइवेट लिमिटेड" is "dream business private limited"). We learned a 1,305-word dictionary from training pairs only. True-pair name similarity went from a median of 0.10 to 1.00. After the first upload scored below validation, test diagnostics showed French records being claimed by several S1 businesses 7× more often than US ones, because many French businesses share one address. Counting how many S1 share each address and street became features.

v4: the cascade

A first LightGBM "filter" scores all 50 candidates per S1 and keeps those with probability ≥ 0.03. Only the survivors go to the final model and into `candidate_pairs.tsv`. Candidates fell 13× (86M → 7.2M on test) and the score did not drop. The filter's training probabilities are out-of-fold, so the final model trains on realistic candidate sets.

v5 and v6: context and a second direction

Name uniqueness: among empty-address candidates with near-identical names, 70% are true matches when exactly one S1 business has that name, 27% when more than 20 do. Reverse competition asks the question from the record's side: which S1 business fits this S2/S3 record best? We searched it against the full S1 file so training and test see the same competition.

v7: bidirectional blocking

The reverse search also *finds* candidates: each record's top 3 S1 businesses are added to those S1 lists. Val recall rose from 0.953 to 0.971. The hub cap of 20 reverse-found candidates per S1 stopped domain-style names from flooding one business.

v8: the transformer

`cross-encoder/ms-marco-MiniLM-L-6-v2` (Apache 2.0, 22.7M parameters) reads both records together as `"name | address" [SEP] "name | address"`. We fine-tuned it on our training pairs in two halves split by S1 entity: each half's model scores only the other half, so its training scores are honest, and test gets the average of both models. On its own it scores 0.961; as one feature of the final LightGBM it lifted holdout from 0.9742 to 0.9794, and the model needed 655 trees instead of 1,822.

v9: deeper reverse search

Each record's top 10 S1 businesses instead of 3. Recall rose to 0.9752, but the extra true pairs are hard cases and only +0.0008 of holdout came through. The v8 transformer

models were reused without retraining; their halves are fixed by a hash of the S1 ID, so the scores stay honest.

05

The final pipeline (v8)

1. **Clean**
   Unicode normalisation, accents stripped, lower case, NULL/N/A removed, look-alike digits repaired (n0rthern → northern), Indian-script words translated with the learned dictionary.

2. **Bidirectional blocking**
   Keys from each record's rarest words (name words, name-word pairs, name prefix, house number × address word, address-word pairs), searched in a packed index per country. Top 50 per S1 forward, plus each record's top 3 S1 from the reverse search, at most 20 reverse-found per S1. Test: 93.4M pairs.

3. **Learned filter**
   LightGBM on about 40 per-pair features, cutoff 0.03. Test: 7.23M pairs, 4.17 per S1. This is candidate_pairs.tsv.

4. **Features + transformer + final LightGBM**
   Name and address similarities, house numbers, group comparisons, S1 context counts, character 3-grams, reverse features and the transformer score.

5. **Decision**
   Each S2/S3 record goes to at most one S1 (its best), then each S1 keeps the subset of candidates that maximises expected F0.5, including the option to predict nothing.

06

The RAM problem, and every step we took

The whole competition ran on an 8 GB MacBook. The data alone is 11M train records and 11.7M test records, and the Apple GPU shares that same 8 GB. Memory decided the design at almost every stage. These are the problems in the order we hit them.

1. Just loading the data

`~6 GB of 8`

PROBLEM — Reading the three big train TSVs together nearly filled the machine, and each TSV took about 6 s to parse.

STEPS — Converted every TSV to Parquet once (compressed, typed, verified identical). Loaded one big file at a time and deleted what we no longer needed. Read only the needed columns, and later only the needed rows (Parquet filters on the entity IDs).

RESULT — 0.4 s per big file and no more whole-dataset loads.

2. Blocking v1

`8.6 GB peak` `→ 3.7 GB`

PROBLEM — The first candidate search built string keys in Python dictionaries: 7.9 GB for only 50k queries. The test set has 1.73M.

STEPS — Words became integer IDs. Each key became a 40-bit hash packed with a 24-bit record number into one 64-bit integer, stored in a single sorted array (lookup is a binary search). Pools loaded per country. S1 processed in batches of 5,000. Compact number types. Keys shared by more than 300 records ignored, since they are too common to help.

RESULT — 3.7 GB peak, about 3.5 minutes on val, and the full test set fit.

3. The official validator

`~10 GB` `fixed`

PROBLEM — With 50 candidates per S1, candidate_pairs.tsv held 86M IDs, and the validator needed about 10 GB to check it.

STEPS — Ran the validator on the scored file only, as its own documentation advises, and wrote `check_candidates.py`, which applies the same rules while reading the file line by line. Killed a duplicate validator process that was doubling the load.

RESULT — Safe checks on 8 GB. After the cascade shrank the file to 7.2M IDs, the full official validator fit again.

4. Training pairs

`15–21M rows`

PROBLEM — Labelled pairs with 40–50 features for train, val and holdout run to tens of millions of rows.

STEPS — Each batch of pairs was written straight to a Parquet file instead of collected in memory. Training loads only the columns a model needs, as 32-bit floats. From v7 on we trained on 80% of the training entities (val and holdout stay complete).

RESULT — v9's 20.7M pairs loaded and trained without running out of memory.

5. The reverse search

`60M links (US train)`

PROBLEM — Searching every S2/S3 record against the full S1 file produces tens of millions of links per country.

STEPS — Scores accumulated in 32-bit numbers, records processed in chunks of 20,000, arrays freed as soon as they were used. The hub cap (20 reverse-found per S1) also removed the 142,645-candidate outlier.

RESULT — Bidirectional blocking ran on the Mac for train, val, holdout and test.

6. Holding test predictions

`small by design`

STEPS — Only pairs that survive the filter are kept, and after scoring only pairs with probability ≥ 0.02 (anything lower can never be selected). Candidate lists are stored as short strings per S1.

7. The transformer overnight

`8.3 GB footprint, 7.1 GB swap`

PROBLEM — The transformer runs on the Apple GPU, which uses the same 8 GB. During the India phase (the largest pool, 4.7M records) the process grew to 8.3 GB and the Mac swapped to disk. Batches slowed from about 45 s to 80–100 s, and scoring ran at about 1,100 pairs/s instead of the 2,170 we measured in training. v8's test run took 6.4 hours instead of 3.5.

STEPS — Kept the Mac on AC power and awake. Ran every stage one after another, never two heavy jobs at once. Tested half precision on the GPU (no gain: 435 vs 548 pairs/s). Stopped all side work once we saw the swapping, and asked to close Safari (about 2.9 GB), Spotify, WhatsApp and MySQL. Wrote a per-country mode for predict.py: one process per country, each country's results saved to disk (written today, not yet run).

RESULT — v8 finished and validated. v9 was slowed and then lost when its parent task was stopped, which the per-country mode is meant to prevent.

8. Looking for a bigger machine

`not used`

STEPS — Prepared a complete package for a friend's 16 GB Windows laptop (setup and run scripts, with UTF-8 fixes). Looked at AWS with the hackathon credits and prepared a v9 package for an AWS GPU server. The GPU quota request is still pending, and credits alone do not allow GPU servers.

RESULT — Everything that was built ran on the 8 GB Mac.

> Smaller problems along the way: a Python set rebuilt inside a list comprehension made two scripts hang (fixed by building it once); background logs looked frozen until we ran Python unbuffered (`-u`); the Mac ran on battery during the first night until it was plugged in.

07

What did not work

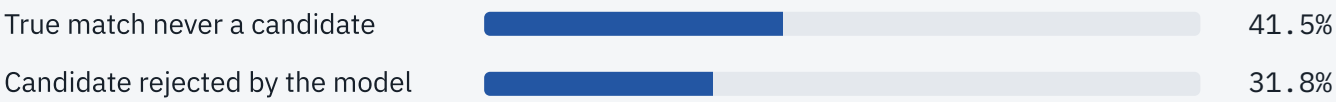
Each of these was tested on val or holdout before we dropped it.

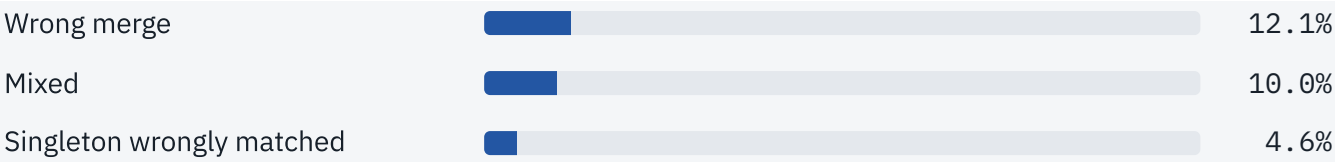
IDEA	RESULT
Ensemble of 3 LightGBMs	Val 0.9660 vs 0.9661 single. No gain.
Whole core-name blocking key	Barely changed recall.
Cheap similarity re-rank of candidates	Worse than the blocking score it replaced.
Raw formatting as a signal	True matches and look-alikes are formatted the same way.
Recalibrating by number-difference type	The model was already calibrated.
Half precision on the Mac GPU	Slower: 435 vs 548 pairs/s.
Reverse top-10 (v9)	Recall +0.0046, holdout only +0.0008.
Sridharan's notebooks as-is	About 0.98, but on a 50k trial subset with 124–179 candidates per S1. On the full pool our blocking at that size also reaches 0.99 recall. The gap came from dataset size.

08

Where the remaining errors are

Error analysis of v7 on val, share of the total lost F0.5 (0.0253):





The biggest bucket is blocking. Even a perfect matcher on v7's candidate lists would score 0.9895 on val, below the leaderboard leader's 0.9907. The largest group of missed pairs (about 37%) are records with an empty address and a common name, where many S1 businesses look identical. The transformer (v8) attacked the second and third buckets; v9 attacked the first.

09

Compute: Mac, laptop, AWS

MacBook M2, 8 GB

Did everything: 9 model versions, 7 full test runs, all overnight chains. GPU (Metal) used for the transformer.

Friend's laptop, 16 GB

Windows, Ryzen 7000, 16 threads, integrated AMD graphics (no CUDA). A ready-to-run package was prepared as a fallback. Not used.

AWS with hackathon credits

Credits pay for servers; the GPU quota decides whether you may start one. New accounts start at 0 and the request is still pending, so no GPU server was possible. Bedrock-style AI APIs were ruled out because the rules ban external APIs.

10

Team inputs

- **Sridharan** shared notebooks 0, 1 and 2. The reverse-competition idea in them became v6b, the biggest single gain of day 2. He also found the AWS credits article and was set to run v9 on AWS.
- **Raghav** raised class imbalance, LightGBM's `num_leaves` and the blocking method, which fed into the v7 review. In the final model the imbalance is handled by the decision layer, tuned directly on val F0.5, and trees use 127 leaves.

- **Arwindh** ran the experiments, made the calls on risk and uploads, and managed the upload budget (three uploads: 0.944, 0.956, 0.967).

11

Where things stand

> **v8 is final and ready.** Validated at 04:57 (PASS on both files), in `output/`, and packaged in `Runtime_Terrors_submission.zip` (352 MB: both outputs, all code, README, pinned requirements, the v8 model with the Apache-2.0 MiniLM, and the methodology document). Holdout 0.9794; our estimate for the leaderboard is about 0.972.

- To finish: upload v8's `matching_results.tsv`, then submit the zip before 23:59. The methodology document still marks v8's leaderboard score as "[UPDATE]".
- v9's model is trained (holdout 0.9802) but has no test outputs, because its run was stopped.
- The per-country change to `predict.py` is written but not tested or committed. It is not in the zip.
- All code is in the local git repository only. Nothing was pushed to the public GitHub account.

12

Lessons

- **Measure speed under real conditions before promising a time.** The transformer's 2,170 pairs/s in training became 1,100 in the full run, and the overnight plan slipped by three hours.
- **Long jobs need checkpoints and must not depend on a UI task.** v9 lost 3.6 hours of work because nothing was saved until the end.
- **Keep the machine idle while a long job runs.** Side work on an 8 GB machine pushes the main job into swap.
- **Arrange bigger compute on day 1.** GPU quota approval takes time; we asked for it on day 3.
- **A clean holdout pays for itself.** It moved in the same direction as the leaderboard every time and let us stop uploading guesses.
- **Recall sets the ceiling.** From v7 on, the candidate lists capped the score more than the matcher did.

- **Read the errors.** France's shared addresses, the planted look-alikes and the "com" hub were all found by looking at real failures, not by tuning.

Runtime Terrors · Amazon ML Challenge 2026 · Compiled 27 Sep 2026 from the team's LOG.md, commit history and run logs.